

Implementation Guide for Improving Disk-LLM

This document outlines concrete improvements for the **disk-LLM** project along with guidance and pseudocode for implementation. Each section includes step-by-step instructions so that a developer can integrate these enhancements into the repository. The overarching goals are to reduce I/O overhead, improve compute efficiency and provide optional configuration knobs.

1. Fuse Q/K/V and MLP projections

Rationale

In the current runtime, each attention layer loads the query, key and value (Q/K/V) weight matrices separately from disk and performs three individual matrix-vector products. Likewise, the feed-forward network (MLP) computes three separate projections (gate/up/down) sequentially. Each operation incurs Python overhead and triggers random disk reads, leading to poor performance ¹. By concatenating the relevant weight matrices into larger blocks at conversion time, a single matrix multiplication can replace multiple dot-products. This improves data locality and allows better utilization of BLAS kernels.

Steps

1. **Modify the converter to fuse projections.** In `converter.py`, after loading a Hugging Face checkpoint for a layer:
2. Load the Q, K and V projection weight matrices.
3. Concatenate them along the first dimension (rows) to form a fused matrix:

```
fused_qkv = np.concatenate([q_proj, k_proj, v_proj], axis=0)
# Optionally concatenate biases if present
fused_bias = np.concatenate([q_bias, k_bias, v_bias], axis=0)
```

- Write `fused_qkv` to the shard file instead of separate Q/K/V entries. Update the manifest to record the starting offset and shape of the fused block and store metadata indicating that the layer uses fused QKV. - Apply the same technique to the MLP (gate/up/down projections) to produce a fused MLP weight (`fused_ggml`), and record it in the manifest.

4. **Change the runtime forward pass.** In `model.py` (or the runtime file for attention), update the function that performs the Q/K/V projection:
5. Read the fused QKV weight from the memmap and reshape it appropriately.
6. Perform a single dot-product with the hidden state (or small batch) to obtain a concatenated result.
7. Slice the result into Q, K and V segments:

```

# Suppose hidden has shape (hidden_size,) and fused_qkv has shape (3 *
qkv_out_dim, hidden_size)
out = hidden @ fused_qkv.T # single GEMM
split = out.shape[0] // 3
q_flat, k_flat, v_flat = np.split(out, 3)
# Reshape into (num_heads, head_dim) as needed
Q = q_flat.reshape(num_heads, head_dim)
K = k_flat.reshape(num_kv_heads, head_dim)
V = v_flat.reshape(num_kv_heads, head_dim)

```

8. Similarly, replace separate MLP projections with a single multiplication using the fused MLP weight, then split the result into gate/up/down parts before applying activation functions.
9. **Maintain backwards compatibility.** Provide environment flags (e.g. `DISK_LLM_FUSE_QKV=1`) that toggle fused behavior. If the flag is not set, fall back to the old separate projection logic to avoid breaking existing models.

Pseudocode summary

```

# during conversion
def fuse_and_write_layer(layer_name, store, manifest, shard_writer):
    q_proj, k_proj, v_proj = store.get(...)
    fused_qkv = np.concatenate([q_proj, k_proj, v_proj], axis=0)
    offset = shard_writer.write(fused_qkv.astype(np.float32))
    manifest[layer_name]['qkv'] = {
        'offset': offset,
        'shape': fused_qkv.shape,
        'dtype': 'float32',
        'fused': True,
    }
# similarly for fused MLP

# during runtime
def compute_attention(layer_meta, hidden):
    fused_qkv = load_mmap(..., offset=layer_meta['qkv']['offset'],
    shape=layer_meta['qkv']['shape'])
    out = np.dot(hidden, fused_qkv.T)
    split = out.shape[0] // 3
    q, k, v = out[:split], out[split:2*split], out[2*split:]
    return q.reshape(...), k.reshape(...), v.reshape(...)

```

2. Batch tokens to exploit matrix-matrix operations

Rationale

Disk-LLM currently processes one token at a time. Each token requires reading the same weights and performing many matrix-vector products, which leads to low throughput. Small batching allows matrix-

matrix multiplication (`np.matmul`) that is more efficient and amortizes overhead across multiple tokens. Care must be taken to keep batch sizes small so that memory usage remains moderate.

Steps

1. **Adjust the generation interface.** Modify the `generate_token_ids` function to accept a batch of input token IDs (e.g. a list of prompts). Preallocate an array `hidden_batch` with shape `(batch_size, hidden_size)` to store the hidden states for each sequence.
2. **Propagate batch dimension through the forward pass.** In each layer, update the code to handle a matrix of hidden states rather than a single vector:
3. Replace `hidden @ weight.T` (shape `(hidden_size,) @ (out_dim, hidden_size).T`) with `hidden_batch @ weight.T`, producing a result of shape `(batch_size, out_dim)`.
4. Similarly update slicing and reshaping logic for Q/K/V and MLP results.
5. Extend the KV cache to have a dimension for batch index and ensure that keys/values are stored per token in the batch.
6. **Maintain incremental decoding.** For autoregressive generation, each token in the batch must have its own sequence length and KV cache offset. Introduce arrays `seq_lengths` and `kv_offsets` to track them. After computing output probabilities, update `hidden_batch` by selecting the newly generated token for each sequence and feed back into the next step.
7. **Set a configurable batch size.** Expose a `--batch_size` flag in the CLI. Default to 1 (current behavior) but allow values up to, for example, 8, depending on available RAM.

Pseudocode summary

```
def generate_token_ids_batch(model, prompts, batch_size):
    seq_lengths = [len(p) for p in prompts]
    hidden_batch = model.embed(prompts) # shape: (batch_size, hidden_size)
    kv_cache = init_kv_cache(batch_size, n_layers, kv_heads, head_dim)
    while not done:
        for layer in layers:
            # use batch matrix multiplication
            qkv_out = hidden_batch @ fused_qkv.T # (batch_size, 3*out_dim)
            q, k, v = split_qkv(qkv_out)
            # update kv cache and compute attention for each batch element
            hidden_batch = attention_step(q, k, v, kv_cache, seq_lengths)
            # apply fused MLP
            mlp_out = hidden_batch @ fused_mlp.T
            gate, up, down = split_mlp(mlp_out)
            hidden_batch = mlp_activation(gate, up, down)
            # sample next tokens for each batch element
            next_tokens = sample_logits(hidden_batch @ lm_head.T)
            # update prompts and hidden states accordingly
    return generated_tokens
```

3. Support lower-precision weights (float16, bfloat16, INT8)

Rationale

Full 32-bit floats occupy twice the disk space of half-precision numbers and require more I/O. Converting weights to `float16` or `bfloat16` cuts disk usage roughly in half and reduces the amount of data read per dot-product. With appropriate care, half-precision arithmetic provides comparable accuracy for inference. More aggressive quantization (INT8) further reduces disk traffic at the cost of additional dequantization logic.

Steps

1. **Extend the converter.** Add a command-line option (e.g. `--dtype={float32,float16,bfloat16,int8}`) to `converter.py`. After loading each weight from the Hugging Face model, call `astype` on the desired dtype before writing to disk:

```
if args.dtype == 'float16':
    weight = weight.astype(np.float16)
elif args.dtype == 'bfloat16':
    weight = weight.astype(np.float32).view(np.uint16) # implement bfloat16
packing
```

For INT8 quantization, compute per-row scale factors and zero-points, quantize the weight to int8, and store the scales/zero-points separately in the manifest.

1. **Update runtime to use specified dtype.** Modify `MemmapTensorStore` to accept a `dtype` parameter and to pass it to `np.memmap` when mapping each shard. For bfloat16 and int8, load the raw data and dequantize on the fly:

```
data = np.memmap(file_path, dtype=np.int8, ...) # for int8
weight = data.astype(np.float32) * scale + zero_point
```

1. **Propagate dtype through computations.** Ensure that arrays used for hidden states, KV cache and intermediate results are created with the appropriate dtype. When performing matrix multiplications, NumPy will upcast smaller dtypes to at least float32 for computation; consider using `np.matmul` with explicit casting to float32 to maintain accuracy.

Pseudocode summary

```
# converter
parser.add_argument('--dtype',
choices=['float32', 'float16', 'bfloat16', 'int8'], default='float32')
weight = weight.astype(dtype_map[args.dtype])
shard_writer.write(weight)

# runtime
def load_weight(meta):
    mm = np.memmap(meta['path'], dtype=meta['dtype'], mode='r',
offset=meta['offset'], shape=meta['shape'])
```

```

if meta['dtype'] == 'int8':
    weight = mm.astype(np.float32) * meta['scale'] + meta['zero_point']
else:
    weight = mm
return weight

```

4. Add optional JIT compilation (Numba)

Rationale

Many critical functions in disk-LLM—such as rotary positional encoding, grouped query attention, and the feed-forward MLP—are implemented in pure Python. Numba's `@njit` decorator can compile these functions into optimized machine code, eliminating Python overhead and accelerating loops.

Steps

1. **Identify candidate functions.** Functions such as `apply_rope_single`, `grouped_query_attention_step`, `gated_delta_step`, and the inner loops of the MLP are good targets for JIT. They operate on NumPy arrays and contain loops that Numba can optimize.
2. **Add Numba dependency (optional).** Update `requirements.txt` to include `numba>=0.58` and conditionally import Numba:

```

try:
    import numba
    NUMBA_AVAILABLE = True
except ImportError:
    NUMBA_AVAILABLE = False

```

1. **Annotate functions with `@njit`.** Wrap target functions with Numba's JIT decorator, and ensure that they operate on statically typed NumPy arrays. For example:

```

from numba import njit

@njit(cache=True)
def apply_rope_single(x, cos, sin):
    for i in range(x.shape[0]):
        xi0 = x[i, 0]
        xi1 = x[i, 1]
        x[i, 0] = xi0 * cos[i] - xi1 * sin[i]
        x[i, 1] = xi0 * sin[i] + xi1 * cos[i]
    return x

```

1. **Create toggles.** Provide an environment flag (e.g. `DISK_LLM_USE_NUMBA=1`) to enable JIT usage. If Numba is unavailable or the flag is unset, fall back to the pure Python implementations.

Pseudocode summary

```
if NUMBA_AVAILABLE and os.getenv('DISK_LLM_USE_NUMBA') == '1':
    @numba.njit(cache=True)
    def apply_rope_single(...):
        ...
else:
    def apply_rope_single(...):
        ... # original implementation
```

5. Use OS-level prefetch hints and larger shard files

Rationale

Currently, disk-LLM's prefetch routine touches each 4 KB page of a layer's weights to trigger the operating system's readahead mechanism ². Grouping layers into larger shard files reduces the number of file descriptors and improves readahead efficiency. Additionally, using the `madvise` system call with flags like `MADV_WILLNEED` or `MADV_SEQUENTIAL` can hint to the OS that sequential access is expected.

Steps

1. **Adjust shard granularity.** In `converter.py`, instead of creating a separate `.bin` file for each layer, group multiple consecutive layers (e.g. 4–8 layers) into a single shard file. Update the manifest so that each tensor's offset is relative to this larger file.
2. **Implement `madvise`.** After opening a mmap for a shard in `MemmapTensorStore`, call `mmap_obj.madvise(mmap.MADV_WILLNEED)` or `mmap_obj.madvise(mmap.MADV_SEQUENTIAL)` to inform the kernel about expected access patterns. On Linux, you can use the `mmap` module's `madvise` function if available. Wrap this in a `try/except` to remain cross-platform.
3. **Tune the prefetch thread.** With larger shards, the explicit page-touching prefetch may no longer be necessary. Allow users to choose between the old method and the OS-level hint via a flag (`DISK_LLM_EXPERIMENT_LAYER_PREFETCH=1` vs `DISK_LLM_USE_MADVISE=1`). Remove or reduce the page-by-page loop if `madvise` suffices.

Pseudocode summary

```
# converter
shard_layers = []
for i, layer_name in enumerate(layer_names):
    shard_layers.append(layer_name)
    if len(shard_layers) == group_size:
        write_shard(shard_layers)
        shard_layers = []
# runtime
mm = np.memmap(path, dtype=dtype, mode='r')
try:
    import mmap
    mm._mmap.madvise(mmap.MADV_WILLNEED)
```

```
except (ImportError, AttributeError):  
    pass
```

6. Cache frequently reused tensors in RAM

Rationale

Some tensors—such as the input embedding matrix and the output projection (LM head)—are accessed on every generation step. Reading them from disk repeatedly is inefficient. Loading these tensors into RAM once and reusing them speeds up inference without sacrificing the disk-backed architecture for larger weights.

Steps

1. **Identify reusable tensors.** Mark the embedding table (`word_embeddings.weight`) and the LM head as cacheable in the manifest. Optionally include positional embeddings and layer norm parameters.
2. **Implement caching flag.** Add a command-line flag `--cache` or environment variable `DISK_LLM_CACHE_TENSORS`. In `MemmapTensorStore.get`, check whether the requested tensor name is marked cacheable and whether caching is enabled. If so, read the memmap into a NumPy array once (`np.array(memmap, copy=True)`) and store it in a dictionary for reuse.
3. **Ensure thread-safety.** Protect the cache dictionary with a lock if inference will run in multi-threaded mode.
4. **Expose a method to clear cache.** Provide a function `clear_cache()` on the tensor store to free memory between runs if necessary.

Pseudocode summary

```
class MemmapTensorStore:  
    def __init__(self, manifest):  
        self.cache = {}  
  
    def get(self, name):  
        meta = self.manifest[name]  
        if meta.get('cacheable') and os.getenv('DISK_LLM_CACHE_TENSORS') ==  
'1':  
            if name not in self.cache:  
                self.cache[name] = np.array(np.memmap(...), copy=True)  
            return self.cache[name]  
        else:  
            return np.memmap(...)
```

7. Provide optional PyTorch or C++ back-ends

Rationale

Even with the above optimizations, pure NumPy may not match the performance of frameworks like PyTorch or specialized C++ implementations (`llama.cpp`). By defining a back-end interface, disk-LLM

can serve as a flexible loader and manifest generator while delegating inference to faster back-ends when desired.

Steps

1. **Define an abstract Runtime class.** Specify methods `initialize_kv_cache`, `forward_step` and `generate`. The current NumPy implementation becomes a subclass `NumPyRuntime`.
2. **Implement TorchRuntime.** Use `torch.from_file` to memory-map the fused weight tensors into PyTorch tensors. Build a `torch.nn.Module` that uses these tensors and implements the same forward logic. Note that PyTorch can perform batched matrix multiplications natively.
3. **Implement a C++/llama.cpp wrapper.** Write a thin Python extension module (using `ctypes` or `cffi`) that reads the manifest, maps the shard files into the format expected by `llama.cpp`, and invokes its C++ inference functions. This allows disk-LLM to convert checkpoints once and run them through `llama.cpp` without duplicating data.
4. **Select runtime at startup.** Add a flag (`--backend={numpy, torch, llama}`) to choose which runtime to instantiate. The converter remains unchanged; only the runtime layer differs.

Pseudocode summary

```
class Runtime:
    def __init__(self, manifest):
        ...
    def forward_step(self, hidden):
        raise NotImplementedError

class NumPyRuntime(Runtime):
    def forward_step(self, hidden):
        ... # existing logic

class TorchRuntime(Runtime):
    def __init__(self, manifest):
        self.qkv_weights = torch.from_file(...)
        ...
    def forward_step(self, hidden):
        hidden = torch.tensor(hidden)
        out = torch.matmul(hidden, self.qkv_weights.T)
        ...

def load_runtime(manifest, backend):
    if backend == 'torch':
        return TorchRuntime(manifest)
    elif backend == 'llama':
        return LlamaRuntime(manifest)
    return NumPyRuntime(manifest)
```


8. Expose configuration knobs for experimentation

Rationale

Disk-LLM is primarily a research tool. Making parameters such as prefetch distance, batch size, dtype and shard size adjustable at runtime encourages experimentation and helps users understand trade-offs.

Steps

1. **Add command-line arguments.** In the CLI script (`cli.py` or `main.py`), use `argparse` to define flags for `--batch_size`, `--dtype`, `--shard_size`, `--use_fused_qkv`, `--use_numba`, `--cache_tensors`, etc. Document these options in the README.
2. **Propagate parameters.** Pass these flags into the converter and runtime constructors so that they can adjust behavior accordingly. For example, the converter uses `dtype` and `shard_size`; the runtime uses `batch_size`, `use_fused_qkv`, and caching flags.
3. **Record telemetry.** When benchmarking, write these configuration values into the output CSV or JSON so that results can be stratified by setting.
4. **Provide default and recommended values.** Include guidelines for typical use cases—for example, recommending `batch_size=4` and `dtype=float16` for machines with limited memory.

Pseudocode summary

```
parser.add_argument('--batch_size', type=int, default=1)
parser.add_argument('--dtype', choices=[...], default='float32')
parser.add_argument('--shard_size', type=int, default=1)
parser.add_argument('--use_fused_qkv', action='store_true')
...
args = parser.parse_args()
converter = Converter(args.dtype, args.shard_size,
    use_fused=args.use_fused_qkv)
runtime = load_runtime(manifest, args.backend)
generated = runtime.generate(prompts, batch_size=args.batch_size)
```

Summary and next steps

Implementing these enhancements will require coordinated changes across the conversion process, manifest format and runtime code. Begin by adding command-line flags and environment variables to enable experimental features. Then, incrementally implement each improvement—starting with fused projections and small batching—while ensuring that unit tests continue to pass. Benchmark each change using the provided telemetry scripts to quantify performance gains before progressing to more complex modifications (e.g. quantization and alternative back-ends).

¹ ² [raw.githubusercontent.com](https://raw.githubusercontent.com/kilickursat/disk-llm/main/src/disk_llm/runtime/model.py)

https://raw.githubusercontent.com/kilickursat/disk-llm/main/src/disk_llm/runtime/model.py